

Laboratorio de Java / Practica 4

MPOO

Esquivel Cortes Iker Alonso

Hernandez Crisostomo Emilio

Ejercicio 1

primerosCinco: Creé y devolví el arreglo con los cinco números en una sola línea, sin necesidad de usar un ciclo for porque los valores ya eran fijos y conocidos desde el principio.

```
// ===== EJERCICIO 1 =====  
// Crea y devuelve un arreglo de int con estos cinco valores, en este orden:  
// 10, 20, 30, 40, 50. Aquí no hace falta ningún for.  
static int[] primerosCinco() {  
    int[] arreglo = {10, 20, 30, 40, 50};  
    return arreglo;  
}
```

Ejercicio 2

promedio: Reutilicé el método suma() para obtener el total del arreglo y lo dividí entre su tamaño. Le agregué un casteo a (double) en la suma para asegurar que la división conservara los decimales y no se truncara como entero.

```
// ===== EJERCICIO 2 =====  
// Devuelve el promedio de los numeros. Ojo con la division entera:  
// si sumas int y divides entre int, Java te devuelve int.  
// Pista: puedes apoyarte en suma(numeros), que ya esta resuelto.  
static double promedio(int[] numeros) {  
    int total = 0;  
    for (int i=0; i<numeros.length; i++){  
        total = total + numeros[i];  
    }  
    return (double) total / numeros.length;  
}
```

Ejercicio 3

máximo: Tomé el primer elemento como candidato a máximo y recorrí los demás con un for tradicional; cada vez que encontraba un valor mayor, actualizaba mi variable candidata.

```
// ===== EJERCICIO 3 =====
// Devuelve el numero mas grande del arreglo.
// Pista: guarda el primero como candidato y recorre el resto comparando.
static int maximo(int[] numeros) {
    int max = numeros[0];
    for (int i = 1; i < numeros.length; i++) {
        if (numeros[i] > max) {
            max = numeros[i];
        }
    }
    return max;
}
```

Ejercicio 4

posicionDe: Recorrí el arreglo con un ciclo for evaluando las posiciones. En cuanto encontré el número buscado regresé de inmediato el índice *i* (garantizando así tomar la primera aparición) y, si terminó el ciclo sin hallarlo, retorné -1.

```
// ===== EJERCICIO 4 =====
// Devuelve la POSICION donde vive el valor buscado, o -1 si no esta.
// Si aparece dos veces, devuelve la primera.
static int posicionDe(int[] numeros, int buscado) {
    for (int i = 0; i < numeros.length; i++) {
        if (numeros[i] == buscado) {
            return i;
        }
    }
    return -1;
}
```

Ejercicio 5

invertido: Instancié un arreglo completamente nuevo del mismo tamaño para no alterar el original. Lo fui llenando de atrás para adelante calculando la posición inversa con $n - 1 - i$.

```
// ===== EJERCICIO 5 =====
// Devuelve un arreglo NUEVO con los mismos valores al revés.
// El arreglo que recibes no se toca.
static int[] invertido(int[] numeros) {
    int n = numeros.length;
    int[] nuevo = new int[n];
    for (int i = 0; i < n; i++) {
        nuevo[i] = numeros[n - 1 - i];
    }
    return nuevo;
}
```

Ejercicio 6

nombreDelMasAlto: Usé la misma lógica de búsqueda de máximo, pero accediendo a la propiedad .calificacion de cada objeto Alumno para comparar y guardando el atributo .nombre del alumno con el valor más alto.

```
// ===== EJERCICIO 6 =====  
// Recibe un arreglo de objetos Alumno y devuelve el nombre del que tiene  
// la calificacion mas alta. La clase Alumno esta hasta abajo.  
// Se llega al dato asi: alumnos[i].nombre    alumnos[i].calificacion  
static String nombreDelMasAlto(Alumno[] alumnos) {  
    int maxCal = alumnos[0].calificacion;  
    String mNombre = alumnos[0].nombre;  
  
    for (int i = 1; i < alumnos.length; i++) {  
        if (alumnos[i].calificacion > maxCal) {  
            maxCal = alumnos[i].calificacion;  
            mNombre = alumnos[i].nombre;  
        }  
    }  
    return mNombre;  
}
```

Ejercicio 7

sumaDeFila: Aproveché que matriz[fila] me da acceso directo al arreglo de esa fila en particular y sumé todos sus elementos usando un ciclo for clásico.

```
// ===== EJERCICIO 7 =====  
// Suma los valores de UNA fila de una matriz.  
// En una matriz, matriz[fila] es a su vez un arreglo: matriz[fila][columna].  
static int sumaDeFila(int[][] matriz, int fila) {  
    int total = 0;  
    for (int valor : matriz[fila]) {  
        total = total + valor;  
    }  
    return total;  
}
```

Ejercicio 8

limpiaYJunta: Separé la cadena en un arreglo usando split(","). Luego recorrí cada elemento para quitarle los espacios sobrantes con .trim() y los fui concatenando, agregando el separador " | " sólo a partir del segundo elemento.

```
// ===== EJERCICIO 8 =====
// Recibe una linea con nombres separados por coma, posiblemente con espacios
// de sobra, y devuelve un solo String con los nombres limpios y separados
// por " | ". " ana , beto ,cris " -> "ana | beto | cris"
// Pistas: linea.split(",") te devuelve un arreglo de String;
//         texto.trim() quita los espacios de las orillas.
static String limpiaYJunta(String linea) {
    String[] partes = linea.split(regex: ",");
    String resultado = "";

    for (int i = 0; i < partes.length; i++) {
        String limpio = partes[i].trim();
        if (i == 0) {
            resultado = limpio;
        } else {
            resultado = resultado + " | " + limpio;
        }
    }
    return resultado;
}
```

Tabla final

```
PS C:\Users\Emilio HC\mpoo\mpoo-2027-1\entregas\hernandez_emilio\p04> javac P4Arreglos.java
PS C:\Users\Emilio HC\mpoo\mpoo-2027-1\entregas\hernandez_emilio\p04> java P4Arreglos
```

```
=====
P4 · ARREGLOS CON TUS MANOS
=====
```

#	ejercicio	esperado	obtuviste	
1	primerosCinco()	[10, 20, 30, 40, 50]	[10, 20, 30, 40, 50]	ok
2	promedio({7,9,5,10,8})	7.8	7.8	ok
3	maximo({7,9,5,10,8})	10	10	ok
4	posicionDe(.., 5)	2	2	ok
5	invertido({7,9,5,10,8})	[8, 10, 5, 9, 7]	[8, 10, 5, 9, 7]	ok
6	nombreDelMasAlto(grupo)	Beto	Beto	ok
7	sumaDeFila(matriz, 1)	60	60	ok
8	limpiaYJunta(linea)	ana beto cris	ana beto cris	ok

```
=====
```

8 DE 8

Listo. Toma la captura de esta tabla y entregala.

```
=====
PS C:\Users\Emilio HC\mpoo\mpoo-2027-1\entregas\hernandez_emilio\p04> □
```